

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
1 АНАЛИТИЧЕСКИЙ РАЗДЕЛ.....	5
1.1 Анализ рынка (Обзор конкурентов и существующих решений)	5
1.1.1 Moodle	5
1.1.2 Google Forms.....	6
1.1.3 Quizizz	6
1.2 Анализ целевой аудитории.....	8
1.2.1 Описание целевой аудитории	8
1.2.2 Проблемы целевой аудитории	9
2 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ	10
2.1. Характеристика веб-приложения	10
2.2. Требования к технологическому решению	13
3. ПРАКТИЧЕСКИЙ РАЗДЕЛ	16
3.1. Фронтенд-разработка	16
3.2. Бэкенд-разработка	21
3.3. Тестирование, оценка и рекомендации	28
ЗАКЛЮЧЕНИЕ	35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36

ВВЕДЕНИЕ

Цифровые инструменты контроля знаний применяются в учебном процессе для проведения текущего, промежуточного и итогового контроля. Электронное тестирование сокращает время проверки, позволяет автоматически фиксировать результаты и даёт преподавателю данные для анализа успеваемости без ручного переноса ответов в таблицы.

В данной работе рассматривается веб-приложение «Система автоматизированного тестирования и самопроверки знаний». Система предназначена для создания, редактирования, прохождения и анализа образовательных тестов. Приложение поддерживает роли студента, преподавателя и администратора, а также разные типы вопросов: один правильный ответ, несколько правильных ответов и сопоставление элементов.

Разрабатываемое приложение относится к клиент-серверным системам. Пользователь работает с одностраничным React-интерфейсом, а операции с данными выполняются через REST API серверной части. Для хранения пользователей, тестов, вопросов, результатов и системных настроек используется PostgreSQL.

Актуальность разработки связана с тем, что готовые сервисы тестирования часто закрывают только часть задач: позволяют быстро создать форму, но не дают гибкого управления ролями, не всегда позволяют хранить данные в собственной базе, имеют ограниченные настройки аналитики и зависят от внешней платформы. Собственное приложение позволяет контролировать модель данных, права доступа, алгоритм проверки ответов и процесс развёртывания.

Целью работы является разработка и развёртывание клиент-серверного веб-приложения для автоматизации создания, прохождения, проверки и анализа образовательных тестов.

Объектом работы является процесс организации электронного тестирования обучающихся. Предметом работы являются методы и средства разработки веб-приложения, реализующего модуль управления тестами и

вопросами, ролевую модель, автоматическую проверку ответов, хранение результатов и аналитику прохождения.

Практическая значимость состоит в том, что результатом работы является полнофункциональная fullstack-система, пригодная для локального запуска через Docker Compose и для размещения в production-среде. Приложение может использоваться преподавателем для подготовки тестов, студентом для прохождения заданий и администратором для сопровождения пользователей и настроек.

Для достижения цели были выполнены следующие задачи:

- проведён анализ предметной области электронного тестирования;
- рассмотрены конкурентные решения Moodle, Google Forms и Quizizz;
- выделены группы пользователей системы и их основные проблемы;
- сформулированы функциональные, нефункциональные и технические требования;
- выбрана клиент-серверная архитектура приложения;
- определён программный стек frontend, backend, базы данных и инфраструктуры;
- спроектированы сущности базы данных и связи между ними;
- реализована клиентская часть на React с маршрутизацией, состоянием Redux и API-сервисом;
- реализована серверная часть на Node.js и Express.js с REST API, JWT-авторизацией и Sequelize;
- реализована ролевая модель для студента, преподавателя и администратора;
- реализованы управление тестами и вопросами, прохождение тестирования и самопроверка знаний;
- реализованы сохранение результатов, повтор ошибок, статистика теста и аналитика преподавателя;

- добавлены экспорт аналитики в PDF и Excel, Swagger-документация и healthcheck-маршрут;
- выполнены модульное, компонентное, API и фаззинг-тестирование;
- подготовлены Dockerfile, Docker Compose, Nginx и CI/CD-пайплайн GitHub Actions;

1 АНАЛИТИЧЕСКИЙ РАЗДЕЛ

В аналитическом разделе рассматривается предметная область электронного тестирования, существующие решения, целевая аудитория и проблемы, которые решает разрабатываемое приложение. Анализ необходим для обоснования состава функций и ограничений системы.

Контроль знаний состоит из подготовки тестовых материалов, предоставления заданий студентам, фиксации ответов, проверки, формирования результата и анализа успеваемости. При ручном подходе часть этих операций выполняется преподавателем вручную, что увеличивает время проверки и повышает риск ошибок при переносе данных.

В цифровом варианте эти этапы можно формализовать: тест хранится в базе данных, вопрос имеет тип и правильный ответ, студент отправляет ответы через интерфейс, backend нормализует данные, вычисляет результат и сохраняет попытку прохождения. После этого преподаватель получает статистику по тесту и по студентам.

1.1 Анализ рынка (Обзор конкурентов и существующих решений)

Для сравнения выбраны три распространённых решения: Moodle, Google Forms и Quizizz. Они отличаются назначением и уровнем сложности: Moodle является полноценной LMS, Google Forms используется для быстрых форм и тестов, Quizizz ориентирован на интерактивные задания и игровую механику.

Сравнение выполнялось по критериям, связанным с фактическими требованиями проекта: создание тестов, поддержка разных типов вопросов, автоматическая проверка, ролевая модель, управление пользователями, аналитика, повтор ошибок, контроль данных.

1.1.1 Moodle

Moodle предоставляет инструменты управления курсами, пользователями, ролями, журналами оценок и тестированием. Система подходит для крупных

образовательных процессов, где требуется полноценная LMS с курсами, группами, материалами и расширяемыми модулями.

Сильными сторонами Moodle являются развитая ролевая модель, поддержка разных типов заданий, хранение данных в собственной инфраструктуре и наличие административных настроек. Для рассматриваемой задачи эти возможности полезны, но часто избыточны: развёртывание, настройка курса и сопровождение Moodle требуют больше времени, чем требуется для компактной системы автоматизированного тестирования.

Недостаток Moodle в контексте данной работы состоит в сложности интерфейса для небольших групп пользователей. Преподавателю, которому нужен быстрая система создания, проверки тестов и анализа результатов, не всегда требуется полный набор функций LMS.

1.1.2 Google Forms

Google Forms позволяет быстро создать форму или тест, добавить варианты ответов, собрать ответы и выгрузить данные. Решение удобно для простых проверок знаний и не требует отдельного развёртывания.

Основное преимущество Google Forms - низкий порог входа: преподаватель может создать тест без настройки сервера и базы данных. При этом решение ограничено при работе с ролевой моделью, административным управлением, повтором ошибок и гибкой аналитикой по структуре теста.

Google Forms зависит от внешнего сервиса и не предоставляет разработчику полного контроля над серверной логикой, алгоритмом проверки, хранением результатов и форматом API.

1.1.3 Quizizz

Quizizz ориентирован на интерактивное тестирование, вовлечение студентов и использование готовых образовательных материалов. Сервис хорошо подходит для игровых сценариев, быстрых проверок и работы с готовыми заданиями.

Сильная сторона Quizizz - удобный пользовательский опыт при прохождении тестов. Однако сервис ограничивает возможность настройки внутренней бизнес-логики, собственной ролевой модели, формата хранения результатов и интеграции с отдельным backend.

Для данной работы важна не только интерактивность, но и управляемость: собственные модели пользователей, тестов, вопросов, результатов, системных настроек, а также возможность тестировать в контролируемой инфраструктуре. Поэтому Quizizz рассматривается как функциональный ориентир, но не как достаточное решение.

Таблица 1.1 - Сравнение существующих решений

Критерий	Moodle	Google Forms	Quizizz	Разрабатываемая система
Создание и редактирование тестов	Да	Да	Да	Да
Поддержка одного правильного ответа	Да	Да	Да	Да
Поддержка нескольких правильных ответов	Да	Частично	Да	Да
Поддержка сопоставления элементов	Да	Ограничено	Да	Да
Автоматическая проверка ответов	Да	Да	Да	Да
Ролевая модель	Да	Нет	Частично	Да
Управление пользователями	Да	Нет	Ограничено	Да
Административные настройки	Да	Нет	Ограничено	Да
Аналитика результатов	Да	Частично	Да	Да
Повтор ошибочных вопросов	Ограничено	Нет	Частично	Да
Экспорт отчётов	Да	Через таблицы	Ограничено	PDF и Excel
Контроль базы данных	Да при self-hosted	Нет	Нет	Да
Контейнеризация и CI/CD в рамках проекта	Требуется настройка	Нет	Нет	Да

По результатам сравнения разрабатываемая система закрывает набор требований, который не полностью покрывается простыми сервисами форм и интерактивных тестов: собственное хранение данных, ролевой доступ, API, административное управление, повтор ошибок, экспорт аналитики.

1.2 Анализ целевой аудитории

Целевая аудитория приложения состоит из трёх групп: студенты, преподаватели и администраторы. Разделение на роли влияет на маршруты frontend, доступные API-операции backend и состав данных, которые пользователь может просматривать или изменять.

1.2.1 Описание целевой аудитории

Студент работает с приложением как с инструментом прохождения тестов. Его сценарий включает регистрацию или вход, просмотр списка доступных тестов, выбор теста, ответы на вопросы, завершение попытки, получение результата и повтор ошибок.

Преподаватель является автором тестов. Его сценарий включает создание теста, задание названия, описания и лимита вопросов, добавление вопросов разных типов, редактирование и удаление вопросов, просмотр статистики конкретного теста, анализ результатов студентов и экспорт отчёта.

Администратор сопровождает платформу. Его сценарий включает просмотр пользователей, изменение ролей, удаление пользователей, просмотр тестов, включение или отключение активности тестов, настройку публичной регистрации, названия платформы и модерации тестов преподавателей.

Разделение функций между ролями уменьшает риск случайного изменения данных. Студент не должен изменять тесты, преподаватель не должен управлять системными настройками, а администратор должен иметь полный контроль над пользователями и состоянием опубликованных тестов.

Таблица 1.2 – Пользователи системы и их рабочие задачи

Роль	Основные задачи	Данные, с которыми работает пользователь	Критичные требования
Студент	Проходить тесты, получать результат, повторять ошибки	Доступные тесты, вопросы, собственные ответы и результаты	Понятный интерфейс, корректный расчёт результата, защита чужих данных
Преподаватель	Создавать тесты, управлять вопросами, анализировать	Собственные тесты, вопросы, результаты студентов,	Модуль управления вопросами, аналитика, экспорт отчётов, защита

	результаты	статистика	тестов автора
Администратор	Управлять пользователями, ролями, тестами и настройками	Пользователи, роли, тесты, системные настройки	Полный контроль, запрет удаления последнего администратора, валидация настроек

1.2.2 Проблемы целевой аудитории

Таблица 1.3 – Пользователи системы и их рабочие задачи

Проблема	Проявление	Решение в приложении
Ручная проверка ответов	Преподаватель тратит время на подсчёт правильных ответов	Backend автоматически проверяет ответы через evaluateAnswers
Разрозненное хранение данных	Результаты и тесты могут храниться в разных файлах и сервисах	Данные хранятся в PostgreSQL в связанных таблицах
Ограниченная аналитика	Сложно быстро увидеть средний процент и проблемные вопросы	Реализованы статистика теста и аналитика преподавателя
Отсутствие ролей	Пользователь может получить доступ к лишним действиям	Маршруты защищены JWT и role middleware
Зависимость от внешних сервисов	Нельзя контролировать backend, API и хранение данных	Приложение разворачивается в собственной контейнеризированной среде
Сложность сопровождения тестов	Редактирование вопросов и вариантов ответа выполняется вручную	Реализован модуль управления вопросами single, multiple и matching

Выявленные проблемы определяют функциональный состав системы. Поэтому в приложении реализованы не только прохождение теста и расчёт балла, но и модуль управления вопросами, ролевая модель, статистика, экспорт отчётов, административное управление.

2 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

Технологический раздел описывает характеристики веб-приложения и требования к реализации. Приложение спроектировано как fullstack-система: frontend предоставляет пользовательский интерфейс, backend реализует бизнес-логику и API, PostgreSQL хранит данные, Docker и Nginx обеспечивают запуск и маршрутизацию сервисов.

2.1. Характеристика веб-приложения

Основным объектом предметной области является веб-приложение для организации образовательного тестирования. Приложение обеспечивает полный цикл работы с тестом: создание, настройку, добавление вопросов, публикацию, прохождение, сохранение результата и анализ.

Система является одностраничным приложением на frontend-стороне. Навигация между страницами выполняется средствами React Router без полной перезагрузки страницы. Это позволяет сохранять состояние авторизации, отображать разные разделы по ролям и быстро переключаться между тестами, профилем, управлением тестами и аналитикой.

Серверная часть предоставляет REST API. Все операции, влияющие на данные, выполняются на backend: регистрация, вход, проверка JWT, проверка роли, создание тестов, управление вопросами, сохранение результата, расчёт статистики и административные действия.

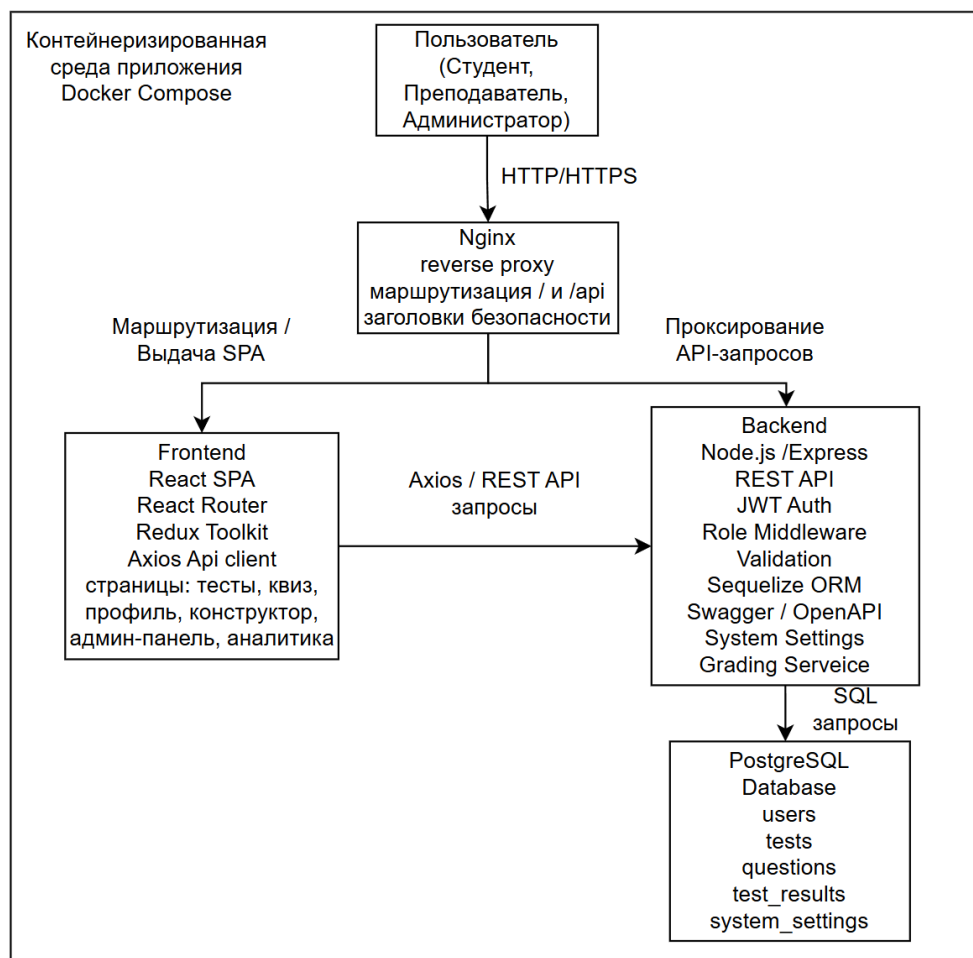


Рисунок 2.1 – Общая архитектура приложения «Система автоматизированного тестирования и самопроверки знаний»

Для хранения данных используется реляционная база PostgreSQL. В проекте определены модели пользователей, тестов, вопросов, результатов и системных настроек. Связи между моделями позволяют получить автора теста, список вопросов, результаты пользователя и попытки прохождения конкретного теста.

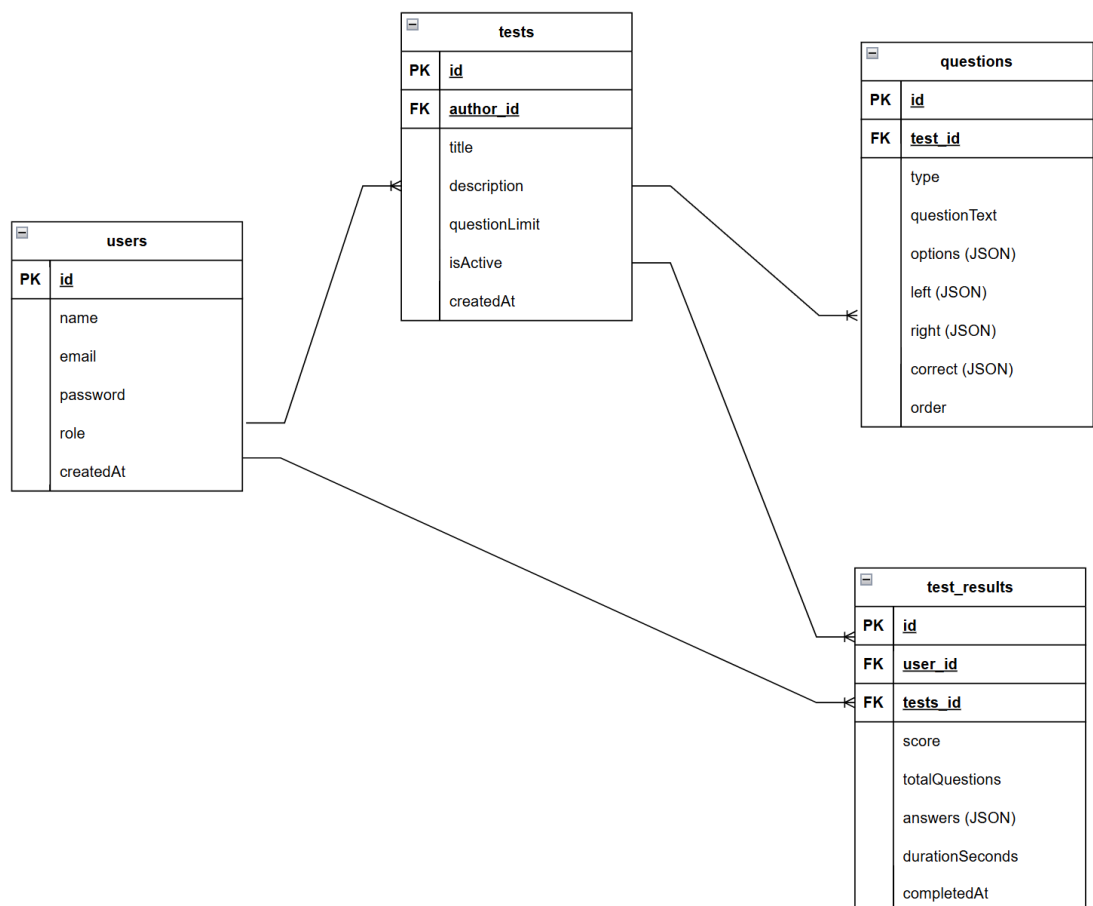


Рисунок 2.2 ER-диаграмма базы данных приложения

В приложении используются три типа вопросов. Для single и multiple вопросов варианты ответа хранятся в JSON-поле options, а правильные индексы - в JSON-поле correct. Для matching вопросов отдельно хранятся массивы left и right, а correct содержит соответствия между элементами.

Результат прохождения включает идентификатор пользователя, идентификатор теста, количество правильных ответов, общее количество вопросов, длительность прохождения, нормализованные ответы и дату завершения. Эти данные используются для профиля студента, повтора ошибок, статистики теста и аналитики преподавателя.

Таблица 2.1 – Основные функции веб-приложения

Группа функций	Реализация	Роль пользователя
Авторизация	Регистрация, вход, получение профиля, хранение JWT	Все пользователи
Прохождение тестов	Список тестов, открытие теста, ответы на вопросы, отправка попытки	Студент и авторизованный пользователь
Управление тестами и	Создание, редактирование, удаление	Преподаватель,

вопросами	тестов и вопросов	администратор
Типы вопросов	single, multiple, matching с отдельной валидацией	Преподаватель, администратор
Результаты	Сохранение попытки, расчёт балла, просмотр истории	Студент
Повтор ошибок	Получение вопросов, на которые пользователь ответил неправильно	Студент, владелец результата, администратор
Статистика теста	Средний балл, средний процент, среднее время, статистика по вопросам	Преподаватель, администратор
Аналитика преподавателя	Группировка по студентам и тестам, экспорт PDF и Excel	Преподаватель
Администрирование	Пользователи, роли, активность тестов, системные настройки	Администратор
Документация и контроль доступности	Swagger/OpenAPI и /api/health	Разработчик, администратор

Таблица 2.2 - Сущности данных приложения

Сущность	Назначение	Ключевые поля
User	Хранение учётных записей пользователей	id, email, password, name, role, createdAt
Test	Хранение тестов и их параметров	id, title, description, questionLimit, authorId, isActive, createdAt
Question	Хранение вопросов теста	id, testId, type, questionText, options, left, right, correct, order
TestResult	Хранение попыток прохождения тестов	id, userId, testId, score, totalQuestions, durationSeconds, answers, completedAt

2.2. Требования к технологическому решению

Технологическое решение должно обеспечивать разделение клиентской и серверной частей. Frontend не должен самостоятельно вычислять критичные права доступа и сохранять данные напрямую в базу. Backend должен проверять пользователя, валидировать входные данные и выполнять операции с моделями.

Frontend должен быть реализован как React SPA. В проекте используются React, React Router, Redux Toolkit, Axios, CSS Modules, pdfmake и xlsx. React отвечает за UI-компоненты, React Router - за маршруты, Redux Toolkit - за состояние авторизации и тестирования, Axios - за HTTP-запросы к API, CSS Modules - за локальные стили, pdfmake и xlsx - за экспорт аналитики.

Backend должен предоставлять REST API на Node.js и Express.js. В проекте используются Express.js, Sequelize, PostgreSQL, JWT, bcryptjs, express-validator,

Swagger и middleware для авторизации и проверки ролей. Sequelize описывает модели и связи, JWT используется для защищённых маршрутов, bcryptjs - для хеширования паролей.

Требования к безопасности включают проверку email и пароля при входе, хеширование паролей перед сохранением, исключение пароля из JSON-ответов пользователя, хранение JWT-секрета в переменных окружения, проверку ролей на сервере и запрет опасных административных операций, например удаления собственной учётной записи или последнего администратора.

Валидация должна выполняться на обеих сторонах. Frontend нормализует данные формы вопроса, убирает пустые варианты и проверяет соответствия matching-вопроса. Backend повторно проверяет длину названия теста, лимит вопросов, идентификаторы, тип вопроса, массив correct, структуру left/right и допустимость роли.

Развёртывание должно быть переносимым. Поэтому в проекте подготовлены Dockerfile клиентской и серверной частей, docker-compose.yml, docker-compose.ubuntu.yml, Nginx-конфигурация, env-файлы и CI/CD workflow. Локально приложение запускается стеком PostgreSQL, backend, frontend и Nginx.

Production-настройки не должны быть зашиты в код. Backend получает PORT, DATABASE_URL, JWT_SECRET, JWT_EXPIRE, NODE_ENV, FRONTEND_URL и CORS_ORIGINS из окружения. Frontend получает REACT_APP_API_URL на этапе сборки, что позволяет направлять клиентские запросы на нужный backend.

Таблица 2.3 - Программный стек проекта

Уровень	Используемые средства	Назначение
Frontend	React, React Router, Redux Toolkit, Axios, CSS Modules	Пользовательский интерфейс, маршрутизация, состояние, HTTP-запросы и стили
Экспорт отчётов	pdfmake,xlsx	Формирование PDF- и Excel-отчётов по аналитике преподавателя
Backend	Node.js, Express.js	REST API и серверная бизнес-логика
Доступ и безопасность	JWT, bcryptjs, express-validator	Токены доступа, хеширование паролей, валидация входных данных
ORM и база данных	Sequelize, PostgreSQL	Модели, связи, хранение пользователей, тестов, вопросов и результатов

Документация	Swagger/OpenAPI	Автоматическое описание маршрутов API
Тестирование	Jest, React Testing Library, Supertest	Модульные, компонентные и API-тесты
Инфраструктура	Docker, Docker Compose, Nginx	Контейнеризация, запуск сервисов, reverse proxy
CI/CD	GitHub Actions, Docker Hub	Автоматическая проверка, сборка и публикация образов
Хостинг	Render	Размещение frontend и backend как отдельных сервисов

Таблица 2.4 - Переменные окружения и назначение

Переменная	Компонент	Назначение
POSTGRES_DB	PostgreSQL	Имя базы данных
POSTGRES_USER	PostgreSQL	Пользователь базы данных
POSTGRES_PASSWORD	PostgreSQL	Пароль пользователя базы данных
DATABASE_URL	Backend	Строка подключения Sequelize к PostgreSQL
JWT_SECRET	Backend	Секрет для подписи JWT-токенов
JWT_EXPIRE	Backend	Срок жизни токена
NODE_ENV	Backend	Режим запуска приложения
PORT	Backend	Порт серверного приложения
FRONTEND_URL	Backend	Разрешённый адрес клиентской части
CORS_ORIGINS	Backend	Список разрешённых источников для CORS
REACT_APP_API_URL	Frontend	Адрес backend API для React-приложения

3. ПРАКТИЧЕСКИЙ РАЗДЕЛ

Практический раздел описывает фактическую реализацию приложения. Основной акцент сделан на клиентской части, серверной части, взаимодействии с API, хранении данных, проверке ответов, тестировании и инфраструктуре запуска.

3.1. Фронтенд-разработка

Клиентская часть реализована в каталоге `client/src`. Точкой входа приложения является `App.js`. В этом файле приложение оборачивается в `Provider Redux store`, настраивается `BrowserRouter` и задаются маршруты основных страниц. При наличии токена и отсутствии загруженного профиля `AppContent` вызывает `fetchProfile`, чтобы восстановить данные пользователя после перезагрузки страницы.

В приложении определены маршруты: `/` для главной страницы, `/tests` для списка тестов, `/test/:testId` для прохождения конкретного теста, `/login` для входа, `/register` для регистрации, `/profile` для профиля, `/analytics` для аналитики преподавателя, `/admin` для административной панели, `/admin/questions` и `/constructor` для управления тестами и вопросами.

Компонент `Header` строит навигацию с учётом авторизации и роли. Ссылка на управление тестами доступна преподавателю и администратору, ссылка на аналитику - преподавателю, ссылка на администрирование - администратору. При выходе вызывается `logout` из `authSlice`, а токен удаляется из состояния и `localStorage`.

Страница `Home` выполняет роль стартовой страницы приложения. Она выводит переходы к основным сценариям: просмотр тестов, регистрация, вход, управление тестами и аналитика в зависимости от состояния авторизации. Страница направляет пользователя к функциональным разделам, а не заменяет рабочий интерфейс.

Страница `Test` получает список тестов через `testsAPI.getAll`. Для каждого

теста отображаются название, описание, количество доступных вопросов и ограничение. При выборе теста пользователь переходит на маршрут прохождения `/test/:testId`.

Прохождение теста реализовано в `QuizContainer`. Компонент загружает тест и вопросы, хранит текущий вопрос, ответы пользователя, вычисляет оставшееся время при наличии лимита и отправляет ответы через `testsAPI.submit`. Для отображения отдельных вопросов используется компонент `Question`, для навигации по тесту - `QuizControls`, для результата и времени - `Score`.

Компонент `Question` поддерживает разные типы вопросов. Для `single` используется выбор одного варианта, для `multiple` - несколько выбранных вариантов, для `matching` - соответствие элементов из левого и правого списков. Изменения ответа передаются в `Redux` через `setUserAnswer`.

Страница `Profile` показывает данные пользователя и историю прохождений. История загружается через `resultsAPI.getMy`. Для результата может быть доступен переход к повтору ошибочных вопросов через `resultsAPI.getMistakes`, что позволяет студенту сосредоточиться на заданиях, выполненных неверно.

Страница `Login` использует `thunk login` из `authSlice`. После успешного входа сервер возвращает JWT и пользователя, а `frontend` сохраняет токен. Страница `Register` использует `thunk register` и отправляет email, пароль и имя пользователя. Публичная регистрация допускает только роль `student`.

Работа с backend вынесена в `client/src/services/api.js`. В `Axios instance` добавляется `baseURL` из `REACT_APP_API_URL` или `http://localhost:5000`, `request interceptor` подставляет `Authorization Bearer token` из `localStorage`, а `response interceptor` обрабатывает ошибку 401 и очищает токен при невалидной авторизации.

Таблица 3.1 - Маршруты клиентского приложения

Маршрут	Компонент	Назначение
/	Home	Стартовая страница и переходы к основным сценариям
/tests	Test	Просмотр списка доступных тестов

/test/:testId	QuizContainer	Прохождение выбранного теста
/login	Login	Авторизация пользователя
/register	Register	Регистрация нового пользователя
/profile	Profile	Профиль и история результатов
/analytics	TeacherAnalytics	Аналитика преподавателя по прохождением
/admin	AdminDashboard	Административная панель
/admin/questions	QuestionsAdmin	Управление тестами и вопросами
/constructor	QuestionsAdmin	Альтернативный маршрут управления тестами для преподавателя

Таблица 3.2 - Основные frontend-компоненты и страницы

Файл или компонент	Ответственность	Фактические операции
App.js	Каркас приложения	Provider, Router, Routes, восстановление профиля, защищённые маршруты.
Header	Навигация	Отображение пунктов меню с учётом авторизации и роли пользователя.
Register, Login, Profile	Учётная запись	Регистрация, вход, сохранение JWT, просмотр профиля и выход.
Home, Test	Работа с тестами	Стартовая страница, список тестов, загрузка выбранного теста.
QuizContainer, Question, QuizControls, Score	Прохождение теста	Показ вопросов, сбор ответов, навигация, отображение результата.
AdminDashboard, QuestionsAdmin	Администрирование	Управление пользователями, ролями, тестами и вопросами.
TeacherAnalytics	Аналитика	Статистика по тестам, результаты студентов, экспорт PDF/Excel.
services/api.js	Интеграция с backend	Axios-клиент, JWT-заголовок, обработка ошибок REST API.
authSlice, quizSlice, store.js	Состояние	Redux Toolkit: пользователь, токен, тесты, текущий тест, результат, ошибки.

Модуль управления тестами и вопросами реализован в QuestionsAdmin.jsx. Компонент доступен только пользователям с ролью teacher или admin. При загрузке вызывается testsAPI.getManageable, затем для выбранного теста загружаются вопросы через testsAPI.getManageQuestions и статистика через resultsAPI.getStats.

Форма вопроса использует структуру createEmptyQuestionForm: id, question, type, options, left, right и correct. Для обычных вопросов по умолчанию создаются четыре варианта ответа. Для matching-вопросов создаются левые и правые списки, а correct хранит соответствие правой части к индексу левой части.

Функция buildQuestionPayload выполняет подготовку данных перед отправкой на backend. Для single и multiple вопросов она обрезает пробелы,

удаляет пустые варианты, пересчитывает индексы правильных ответов и проверяет наличие минимум двух вариантов. Для matching вопросов она проверяет заполненность списков, корректность индексов, длину соответствий и уникальность использования левой части.

При редактировании существующего вопроса используется `normalizeQuestionForForm`. Она преобразует данные backend-модели `Question` в состояние формы frontend: `questionText` становится `question`, массивы `options`, `left` и `right` дополняются до минимальной длины, а `correct` переносится без потери индексов.

В модуле управления тестами реализованы создание теста, редактирование названия и описания, настройка `questionLimit`, удаление теста, добавление вопроса, редактирование вопроса и удаление вопроса. Если тест содержит вопросы, перед удалением выводится подтверждение, так как вопросы будут удалены вместе с тестом.

Таблица 3.3 - Подготовка данных формы вопроса на frontend

Тип вопроса	Поля формы	Проверки перед отправкой	Отправляемая структура
single	question, type, options, correct	Минимум два непустых варианта, минимум один правильный ответ	question, type, options, correct
multiple	question, type, options, correct	Минимум два непустых варианта, один или несколько правильных ответов	question, type, options, correct
matching	question, type, left, right, correct	Все пары заполнены, индексы корректны, каждая левая часть используется один раз	question, type, left, right, correct

Административная панель `AdminDashboard` доступна только пользователю с ролью `admin`. При загрузке панель параллельно получает пользователей, тесты и системные настройки через `adminAPI.getUsers`, `adminAPI.getTests` и `adminAPI.getSettings`. Если пользователь не является администратором, запросы не выполняются.

В административной панели реализованы изменение роли пользователя, удаление пользователя, включение или отключение активности теста и

изменение системных настроек. Для отображения пользователей используются счётчики `testCount` и `resultCount`, полученные с backend. Для тестов отображаются `author`, `questionCount`, `totalAttempts` и `averageScore`.

Аналитика преподавателя реализована в `TeacherAnalytics.jsx`. Данные загружаются через `resultsAPI.getTeacherPerformance`. Компонент нормализует отчёт, группирует результаты по студентам или тестам, вычисляет средний процент, количество правильных и неправильных ответов, среднее время прохождения и выводит детали ответов.

Для экспорта аналитики используется `xlsx` и `pdfmake`. Excel-экспорт формирует отдельные листы со сводкой, результатами и ответами. PDF-экспорт формирует структурированный отчёт с данными по студентам, тестам, процентам, времени прохождения и деталям ответов. Перед выводом в PDF текст нормализуется: символ стрелки заменяется на ASCII-представление, чтобы избежать проблем со шрифтами.

Таблица 3.4 - Методы API-сервиса frontend

Группа	Метод	HTTP-запрос
<i>authAPI</i>	<i>register, login, fetchProfile</i>	<i>POST /api/auth/register; POST /api/auth/login; GET /api/auth/me</i>
<i>testsAPI</i>	<i>getTests, getTest, createTest, updateTest, deleteTest</i>	<i>GET /api/tests; GET/PUT/DELETE /api/tests/:id; POST /api/tests</i>
<i>questionAPI</i>	<i>createQuestion, updateQuestion, deleteQuestion</i>	<i>POST /api/tests/:testId/questions; PUT/DELETE /api/questions/:id</i>
<i>resultsAPI</i>	<i>submitTest, getMyResults, getTestStats</i>	<i>POST /api/tests/:id/submit; GET /api/results/my-results; GET /api/tests/:id/stats</i>
<i>adminAPI</i>	<i>getUsers, updateUserRole, deleteUser</i>	<i>GET /api/admin/users; PATCH/DELETE /api/admin/users/:id</i>
<i>settingsAPI</i>	<i>getSettings, updateSettings</i>	<i>GET /api/admin/settings; PUT /api/admin/settings</i>

Состояние frontend разделено на независимые slices. `authSlice` отвечает за регистрацию, вход, загрузку профиля, выход и хранение ошибки. Начальное состояние получает токен из `localStorage`, чтобы пользователь оставался авторизованным после обновления страницы.

`quizSlice` отвечает за загрузку теста, хранение списка вопросов, ответы пользователя, отправку результата, ошибки и состояние загрузки. Такое

разделение упрощает тестирование: authSlice можно проверять отдельно от прохождения теста, а quizSlice - отдельно от страниц входа и регистрации.

Таблица 3.5 - Redux-состояние клиентской части

Slice	Ключевые поля	Основные действия
authSlice	user, token, isAuthenticated, loading, error	register, login, fetchProfile, logout, clearError
quizSlice	currentQuiz, questions, userAnswers, score, loading, error	fetchQuiz, submitQuiz, setUserAnswer, resetQuiz

Фронтенд покрыт компонентными и модульными тестами. В исходниках присутствуют тесты App, Header, Question, QuizContainer, QuizControls, Score, AdminDashboard, QuestionsAdmin, Home, Login, Profile, Register, TeacherAnalytics, Test, api.js, authSlice и quizSlice. Для компонентов используется React Testing Library, для slices - проверка редьюсеров и async thunk, для API-сервиса - мокирование Axios.

3.2. Бэкенд-разработка

Серверная часть реализована в каталоге server/src. Основной файл app.js подключает Express, CORS, dotenv, sequelize, маршруты auth, tests, results и admin, Swagger-документацию, а также сервис ensureDefaultSettings. При запуске backend проверяет соединение с базой данных, синхронизирует модели Sequelize и создаёт настройки по умолчанию.

В app.js настроены middleware express.json и cors. CORS допускает локальные адреса разработки и значения из переменных FRONTEND_URL и CORS_ORIGINS. Это позволяет запускать приложение локально и в production-среде, не меняя код backend.

Backend предоставляет маршрут /api/health. Он возвращает статус ok, timestamp и environment. Маршрут используется для проверки работоспособности в Docker Compose, GitHub Actions и при внешнем мониторинге.

Структура backend разделена на контроллеры, маршруты, middleware, модели, сервисы и утилиты. Такой подход делает код сопровождаемым:

маршруты отвечают за адреса и валидацию, контроллеры - за бизнес-логику, модели - за структуру таблиц, middleware - за доступ, а утилиты - за повторно используемые алгоритмы.

В качестве ORM используется Sequelize. Модели определяют таблицы users, tests, questions, test_results и system_settings. Связи задаются в models/index.js: Test принадлежит User как author, User имеет много Test, Question принадлежит Test, Test имеет много Question, TestResult принадлежит User и Test.

Пользовательская модель User содержит email, password, name, role и createdAt. Перед созданием и обновлением пользователя пароль хешируется с помощью bcryptjs. Метод comparePassword сравнивает введенный пароль с хешем, а toJSON исключает password из ответа.

Модель Test хранит title, description, questionLimit, authorId, isActive и createdAt. Название теста валидируется по длине от 5 до 200 символов, questionLimit должен быть положительным числом, authorId связывает тест с пользователем-автором.

Модель Question хранит type, questionText, options, left, right, correct и order. Поле type ограничено значениями single, multiple и matching. Поля options, left, right и correct имеют тип JSON, что позволяет хранить разные структуры данных для разных типов вопросов.

Модель TestResult хранит userId, testId, score, totalQuestions, durationSeconds, answers и completedAt. Поле answers хранит нормализованные ответы пользователя, которые затем используются для повтора ошибок и аналитики.

Таблица 3.6 - Модели Sequelize и поля

Модель	Таблица	Поля	Особенности
User	users	id, email, password, name, role, createdAt	email уникален, role: student/teacher/admin, пароль хешируется
Test	tests	id, title, description, questionLimit, authorId, isActive, createdAt	authorId связан с users, isActive управляет доступностью

Question	questions	id, testId, type, questionText, options, left, right, correct, order	type: single/multiple/matching, ответы хранятся в JSON
TestResult	test_results	id, userId, testId, score, totalQuestions, durationSeconds, answers, completedAt	answers хранит нормализованные ответы

Авторизация реализована в `authController`. Метод `register` проверяет настройку публичной регистрации, создаёт пользователя и возвращает JWT. Метод `login` ищет пользователя по `email`, сравнивает пароль через `comparePassword` и возвращает токен. Метод `getProfile` отдаёт данные текущего пользователя без пароля.

Маршруты авторизации определены в `routes/auth.js`. `POST /register` проверяет `email`, пароль длиной не менее 6 символов, имя длиной от 1 до 100 символов без HTML-тегов и запрещает публичную регистрацию с ролью выше `student`. `POST /login` проверяет формат `email` и наличие пароля. `GET /profile` защищён `middleware auth`.

`Middleware auth` извлекает `Bearer token` из заголовка `Authorization`, проверяет его через `JWT_SECRET`, находит пользователя в базе данных и записывает его в `req.user`. Если токен отсутствует или невалиден, возвращается ошибка доступа.

`Middleware role` принимает список допустимых ролей. Если `req.user` отсутствует или его роль не входит в разрешённый список, `backend` возвращает 403. Этот `middleware` используется для маршрутов преподавателя, администратора и операций управления тестами.

`Middleware validate` обрабатывает результат `express-validator`. Если в запросе есть ошибки валидации, `backend` возвращает 400 и массив ошибок. Это обеспечивает единый формат ответа для некорректных параметров, тела запроса и идентификаторов.

Таблица 3.7 - *Middleware backend*

Middleware	Назначение	Где применяется
auth	Проверяет JWT, загружает пользователя и записывает <code>req.user</code>	Профиль, результаты, управление тестами, admin API

optionalAuth	Пытается загрузить пользователя, но допускает запрос без токена	Просмотр теста и отправка теста, где возможен публичный сценарий
role	Проверяет принадлежность пользователя к разрешённым ролям	Управление тестами, статистика, аналитика, admin API
validate	Возвращает ошибки express-validator единым ответом	Все маршруты с body или param validation

Маршруты tests отвечают за получение тестов, управление тестами, получение вопросов и отправку ответов. Публичный список тестов доступен через GET /api/tests. Управляемые тесты автора доступны через GET /api/tests/manage только для teacher и admin. Получение вопросов для прохождения доступно через GET /api/tests/:id/questions, а служебное получение вопросов для редактирования - через GET /api/tests/:id/questions/manage.

Создание теста выполняется через POST /api/tests и доступно преподавателю или администратору. Backend проверяет название теста, массив questions и questionLimit. Редактирование теста выполняется через PUT /api/tests/:id, удаление - через DELETE /api/tests/:id. Для каждого маршрута проверяется id теста и право пользователя управлять этим тестом.

Маршруты вопросов вложены в tests. Создание вопроса выполняется через POST /api/tests/:id/questions, редактирование - через PUT /api/tests/:id/questions/:questionId, удаление - через DELETE /api/tests/:id/questions/:questionId. Для вопроса проверяется текст длиной от 5 до 500 символов, тип, correct, options или left/right в зависимости от типа.

Для matching-вопроса backend проверяет наличие левого и правого списков, непустые строки, одинаковую длину списков, корректность индексов correct и уникальность использования каждой левой части. Это дублирует frontend-проверки и защищает API от некорректных запросов напрямую.

Отправка теста выполняется через POST /api/tests/:id/submit. Backend получает ответы, загружает вопросы теста, применяет questionLimit при необходимости, нормализует длительность прохождения, вычисляет результат через evaluateAnswers и возвращает score, totalQuestions и incorrectQuestionIds.

Таблица 3.8 - Основные backend-маршруты

Метод и путь	Контроллер	Доступ	Назначение
POST /api/auth/register	authController.register	Публичный	Создание пользователя student и выдача JWT.
POST /api/auth/login	authController.login	Публичный	Проверка учётных данных и выдача JWT.
GET /api/auth/me	authController.getMe	Авторизованный пользователь	Получение текущего профиля.
GET /api/tests; GET /api/tests/:id	testController	Все пользователи	Список тестов и детальная структура теста.
POST /api/tests; PUT/DELETE /api/tests/:id	testController	teacher/admin	Создание, изменение и удаление тестов.
POST /api/tests/:testId/questions	questionController.createQuestion	teacher/admin	Добавление вопроса выбранного типа.
PUT/DELETE /api/questions/:id	questionController	teacher/admin	Редактирование и удаление вопроса.
POST /api/tests/:id/submit	resultController.submitTest	student/teacher/admin	Проверка ответов и сохранение попытки.
GET /api/results/my-results	resultController.getMyResults	Авторизованный пользователь	История результатов пользователя.
GET /api/tests/:id/stats	resultController.getTestStats	teacher/admin	Статистика теста для преподавателя.
GET/PATCH/DELETE /api/admin/users/:id	adminController	admin	Управление пользователями и ролями.
GET/PUT /api/admin/settings	adminController	admin	Системные настройки приложения.
GET /health; GET /api-docs	app.js, swagger.js	Публичный	Healthcheck и Swagger-документация.

Расчёт результатов вынесен в server/src/utils/grading.js. Утилита нормализует ответы, удаляет дубликаты, преобразует значения к целым числам

и сортирует массивы ответов. Это важно для multiple-вопросов, где порядок выбора не должен влиять на результат.

Для single и multiple вопросов применяется сравнение массивов: ответ и правильные индексы приводятся к отсортированным массивам целых чисел, после чего сравниваются по длине и значениям. Для matching-вопросов ответ приводится к объекту, где ключом является индекс правого элемента, а значением - индекс левого элемента. Далее объект сравнивается с массивом correct.

Функция evaluateAnswers получает список вопросов и массив ответов. Она строит Map ответов по questionId, проходит по каждому вопросу, увеличивает score при правильном ответе и добавляет id вопроса в incorrectQuestionIds при ошибке. Возвращаемый объект содержит score, totalQuestions, incorrectQuestionIds и нормализованные answers.

Таблица 3.9 - Алгоритм проверки ответов

Этап	Действие	Результат
Нормализация id	questionId и индексы приводятся к integer	Некорректные значения отбрасываются
Нормализация массива	Удаляются дубликаты, значения сортируются	Порядок выбора не влияет на multiple
Нормализация matching	Ответ преобразуется к объекту rightIndex -> leftIndex	Можно сравнить пары соответствий
Сравнение single/multiple	Сравниваются нормализованный ответ и correct	Определяется правильность обычного вопроса
Сравнение matching	Сравнивается соответствие каждого rightIndex	Определяется правильность matching-вопроса
Подсчёт результата	Выполняется проход по вопросам	Возвращаются score и incorrectQuestionIds

Контроллер resultController отвечает за сохранение результатов, историю пользователя, повтор ошибок, статистику теста и аналитику преподавателя. Метод saveResult проверяет существование теста, загружает вопросы, сверяет переданный набор questionId, вычисляет результат и создаёт TestResult.

Метод getResultMistakes проверяет, что результат существует, а пользователь имеет право его просматривать. Результат может смотреть владелец, администратор или пользователь, который может управлять тестом. После проверки backend повторно вычисляет incorrectQuestionIds и отдаёт

только вопросы, на которые был дан неправильный ответ.

Метод `getTestStatistics` используется для статистики конкретного теста. Он загружает вопросы и результаты, вычисляет `totalAttempts`, `averageScore`, `averagePercent`, `averageDurationSeconds` и статистику по каждому вопросу: `correctCount`, `incorrectCount`, `totalAnswers` и `correctPercent`.

Метод `getTeacherPerformance` строит отчёт по тестам преподавателя. Он получает все тесты автора, загружает вопросы и результаты по этим тестам, связывает результаты со студентами и тестами, вычисляет корректные и некорректные ответы, проценты, длительность и детали ответов. Эти данные затем используются frontend-страницей `TeacherAnalytics` для группировки и экспорта.

Административный контроллер возвращает пользователей со счётчиками созданных тестов и результатов, тесты со статистикой вопросов и попыток, обновляет роли пользователей, удаляет пользователей и обновляет системные настройки. При удалении пользователя backend запрещает удалить самого себя и последнего администратора.

Удаление пользователя выполняется в транзакции `Sequelize`. Сначала удаляются результаты пользователя, затем тесты удаляемого автора передаются текущему администратору, после чего удаляется сам пользователь. Такой порядок сохраняет тестовые материалы и предотвращает потерю тестов при удалении автора.

Swagger-документация подключается через `setupSwagger`. Она используется для описания REST API и удобной проверки маршрутов разработчиком. Healthcheck-маршрут даёт простой JSON-ответ, пригодный для автоматических проверок доступности сервиса.

База данных и backend запускаются в Docker Compose. Backend зависит от PostgreSQL и стартует после успешного healthcheck базы данных через `pg_isready`. Это исключает запуск Node.js-сервера до готовности PostgreSQL принимать подключения.

Nginx выполняет роль reverse proxy. В локальном `docker-compose.yml`

frontend доступен на порту 3001, backend - на 5000, а Nginx - на 80. Запросы к пользовательскому интерфейсу обслуживаются frontend-сервисом, а запросы с /api проксируются на backend

3.3. Тестирование, оценка и рекомендации

Тестирование приложения выполнялось на уровне frontend-компонентов, Redux-логики, API-сервиса, backend-контроллеров, middleware, утилит проверки ответов и инфраструктуры запуска. Такой набор проверок соответствует структуре приложения и покрывает основные пользовательские сценарии.

Frontend-тесты используют Jest и React Testing Library. Проверяются отображение навигации по ролям, работа форм входа и регистрации, отображение профиля, прохождение теста, компоненты вопроса, кнопки управления тестом, расчёт результата, модуль управления вопросами, административная панель и аналитика преподавателя.

```
PS C:\Users\relax\Documents\kursach_6_PiRKSP\client> npm run coverage

> test-constructor-client@1.0.0 coverage
> react-scripts test --coverage --watchAll=false --runInBand

PASS src/App.test.js
PASS src/pages/TeacherAnalytics/TeacherAnalytics.test.jsx
PASS src/pages/Admin/QuestionsAdmin.test.jsx
PASS src/pages/Admin/AdminDashboard.test.jsx
PASS src/components/Question/Question.test.jsx
PASS src/components/Quiz/QuizContainer.test.jsx
PASS src/pages/Register/Register.test.jsx
PASS src/pages/Profile/Profile.test.jsx
PASS src/pages/Login/Login.test.jsx
PASS src/pages/Test/Test.test.jsx
PASS src/components/Header/Header.test.jsx
PASS src/pages/Home/Home.test.jsx
PASS src/components/Score/Score.test.jsx
PASS src/components/QuizControls/QuizControls.test.jsx
PASS src/store/slices/quizSlice.test.js
PASS src/store/slices/authSlice.test.js
PASS src/services/api.test.js

-----
File                                     % Stmts   % Branch   % Funcs   % Lines   Uncovered Line #s
-----
All files                               100        100        100        100
src                                     100        100        100        100
  App.js                               100        100        100        100
src/components/Header                  100        100        100        100
  Header.jsx                           100        100        100        100
src/components/Question                100        100        100        100
  Question.jsx                         100        100        100        100
src/components/Quiz                    100        100        100        100
  QuizContainer.jsx                    100        100        100        100
src/components/QuizControls            100        100        100        100
  QuizControls.jsx                     100        100        100        100
src/components/Score                   100        100        100        100
  Score.jsx                            100        100        100        100
src/pages/Admin                       100        100        100        100
  AdminDashboard.jsx                   100        100        100        100
src/pages/Home                         100        100        100        100
  Home.jsx                             100        100        100        100
src/pages/Login                       100        100        100        100
  Login.jsx                            100        100        100        100
src/pages/Profile                     100        100        100        100
  Profile.jsx                          100        100        100        100
src/pages/Register                    100        100        100        100
  Register.jsx                         100        100        100        100
src/pages/TeacherAnalytics             100        100        100        100
  TeacherAnalytics.jsx                 100        100        100        100
src/pages/Test                        100        100        100        100
  Test.jsx                             100        100        100        100
src/services                          100        100        100        100
  api.js                               100        100        100        100
src/store                              100        100        100        100
  store.js                             100        100        100        100
src/store/slices                      100        100        100        100
  authSlice.js                         100        100        100        100
  quizSlice.js                         100        100        100        100
-----

Test Suites: 17 passed, 17 total
Tests:       82 passed, 82 total
Snapshots:   0 total
Time:        10.353 s, estimated 63 s
Ran all test suites.
```

Рисунок 3.1 - Успешное выполнение компонентных тестов клиентской части

Backend-тесты используют Jest и Supertest. Проверяются маршруты авторизации, middleware auth, optionalAuth, role и validate, утилита grading, контроллеры тестов и результатов, административные операции, системные настройки, Swagger и healthcheck.

```

PS C:\Users\relax\Documents\kursach_6_PiRKSP\server> npm run coverage

> test-constructor-server@1.0.0 coverage
> node ../client/node_modules/jest/bin/jest.js --coverage --runInBand

PASS tests/swagger.test.js
PASS tests/middleware.test.js
PASS tests/systemSettings.test.js
PASS tests/roles.test.js
PASS tests/grading.test.js

-----|-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|
All files |    100 |    100 |    100 |    100 |
constants |    100 |    100 |    100 |    100 |
  roles.js |    100 |    100 |    100 |    100 |
middleware |    100 |    100 |    100 |    100 |
  auth.js  |    100 |    100 |    100 |    100 |
  optionalAuth.js |    100 |    100 |    100 |    100 |
  role.js  |    100 |    100 |    100 |    100 |
  validate.js |    100 |    100 |    100 |    100 |
services  |    100 |    100 |    100 |    100 |
  systemSettings.js |    100 |    100 |    100 |    100 |
utils     |    100 |    100 |    100 |    100 |
  grading.js |    100 |    100 |    100 |    100 |
-----|-----|-----|-----|-----|

Test Suites: 5 passed, 5 total
Tests:       29 passed, 29 total
Snapshots:   0 total
Time:        1.633 s, estimated 2 s
Ran all test suites.
PS C:\Users\relax\Documents\kursach_6_PiRKSP\server>

```

Рисунок 3.2 - Успешное выполнение тестов серверной части

```

PS C:\Users\relax\Documents\kursach_6_PiRKSP> curl http://localhost:5000/api/health

StatusCode      : 200
StatusDescription : OK
Content         : {"status":"OK","timestamp":"2026-05-04T19:43:28.482Z","environment":"development"}
RawContent      : HTTP/1.1 200 OK
                  Access-Control-Allow-Origin: *
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Content-Length: 82
                  Content-Type: application/json; charset=utf-8
                  Date: Mon, 04 May 2026 19:43:28 GMT
                  ...
Forms           : {}
Headers         : {[Access-Control-Allow-Origin, *], [Connection, keep-alive], [Keep-Alive, timeout=5], [Content-Length, 82]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 82

```

Рисунок 3.3 - Проверка healthcheck-маршрута backend

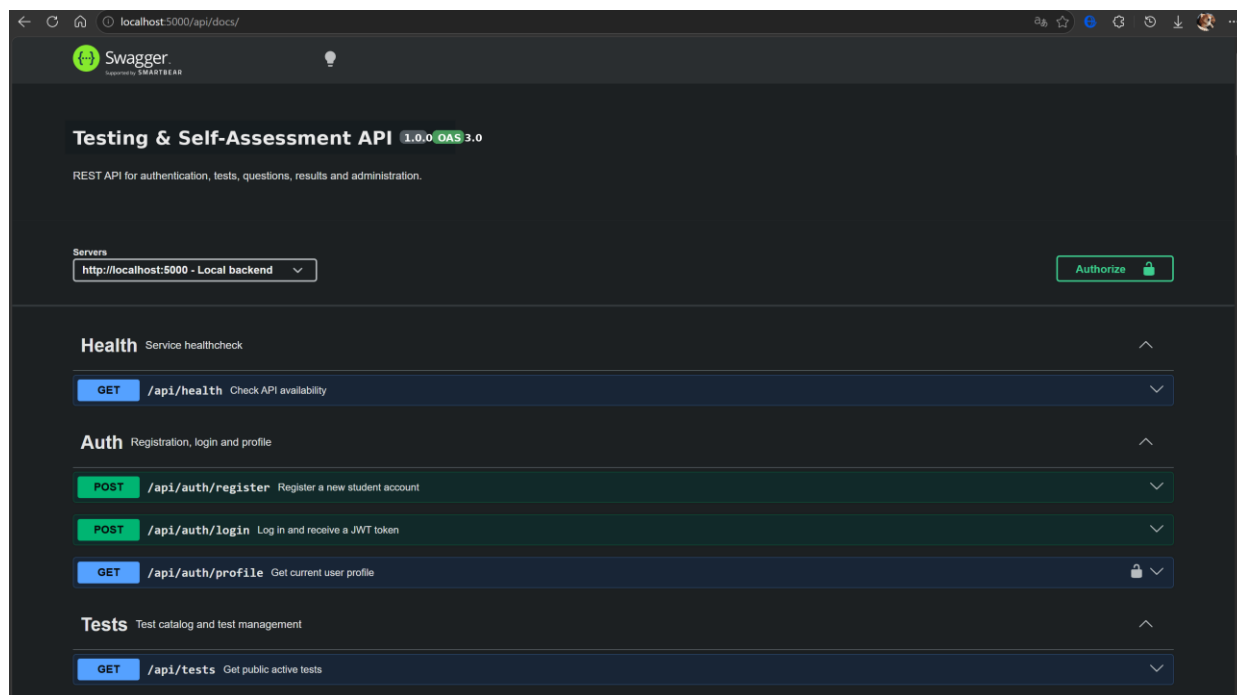


Рисунок 3.4 - Swagger-документация API

Фаззинг-тестирование API выполнялось с использованием некорректных payload, вредоносных строк, неправильных идентификаторов, попыток доступа с разными ролями и нестандартных значений. В результате проверялась устойчивость валидации и корректность статусов 400, 401, 403 и 404.

```

PS C:\Users\relax\Documents\kursach_6_PiRKSP> node .\server\tests\fuzzing\fuzzing.js
Starting fuzzing tests...

Target: http://localhost:5000

Testing SQL injection...

PASS SQL Injection Login -> /api/auth/login (400)
PASS SQL Injection Route -> /api/tests/'%20OR%20'1'%3D'1 (400)
PASS SQL Injection Login -> /api/auth/login (400)
PASS SQL Injection Route -> /api/tests/'%3B%20DROP%20TABLE%20users%3B-- (400)
PASS SQL Injection Login -> /api/auth/login (400)
PASS SQL Injection Route -> /api/tests/'%20UNION%20SELECT%20*%20FROM%20users-- (400)
PASS SQL Injection Login -> /api/auth/login (401)
PASS SQL Injection Route -> /api/tests/admin'-- (400)
PASS SQL Injection Login -> /api/auth/login (400)
PASS SQL Injection Route -> /api/tests/'%20OR%201'%3D1-- (400)

Testing XSS payload handling...

PASS XSS Register Payload -> /api/auth/register (400)
PASS XSS Register Payload -> /api/auth/register (400)
PASS XSS Register Payload -> /api/auth/register (400)

Testing authentication guards...

PASS Protected Profile Access -> /api/auth/profile (401)
PASS Protected Profile Access -> /api/auth/profile (401)
PASS Protected Profile Access -> /api/auth/profile (401)
PASS Protected Profile Access -> /api/auth/profile (401)

Testing path traversal, command injection and null bytes...

PASS Traversal/Command Route Probe -> /api/tests/..%2F..%2Fetc%2Fpasswd/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/..%5C..%5C..%5Cwindows%5Csystem32%5Cconfig%5Csam/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%252e%252e%252f%252e%252e%252f%252e%252e%252fetc%252fpasswd/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%3B%20cat%20%2Fetc%2Fpasswd/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%26%26%20whoami/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%24(id)/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%00/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/test%00admin/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%2500/questions (400)

Testing large payloads...

PASS Large Payload Register -> /api/auth/register (400)
PASS Large Payload Register -> /api/auth/register (400)
PASS Large Payload Register -> /api/auth/register (400)

Testing RBAC...

PASS Auth Login -> /api/auth/login (200)
PASS Auth Login -> /api/auth/login (200)
PASS Auth Login -> /api/auth/login (200)
PASS RBAC Create Test (admin) -> /api/tests (201)
PASS RBAC Delete Test (admin) -> /api/tests/42 (200)
PASS RBAC Create Test (teacher) -> /api/tests (201)
PASS RBAC Delete Test (teacher) -> /api/tests/43 (200)
PASS RBAC Create Test (student) -> /api/tests (403)

Results saved to: C:\Users\relax\Documents\kursach_6_PiRKSP\server\tests\fuzzing\results\fuzzing-report-1777923996527.json
Passed checks: 37
Failed checks: 0

```

Рисунок 3.5 - Успешное выполнение фаззинг-тестирования API

Инфраструктурная проверка выполняется через GitHub Actions. Workflow запускается при push и pull request в main/master, а также вручную. Сначала устанавливаются зависимости client и server, затем запускаются тесты, после этого собирается Docker Compose stack и проверяются backend, frontend и Nginx.

CI/CD также предусматривает публикацию Docker-образов frontend и backend в Docker Hub. Для этого используются docker/build-push-action, секреты DOCKERHUB_USERNAME и DOCKERHUB_TOKEN, а образы получают теги latest и SHA текущего коммита.

Таблица 3.11 - Проверяемые сценарии тестирования

Группа тестов	Что проверяется	Инструменты
Авторизация	Регистрация, вход, профиль, ошибки валидации	Jest, Supertest, React Testing Library
Просмотр тестов	Загрузка списка и открытие карточки теста	React Testing Library, мок API
Прохождение теста	Ответы single/multiple/matching, расчёт результата	RTL, модуль grading.js
Повтор ошибок	Формирование набора неверных ответов без перезаписи результата	RTL, ручная проверка сценария
Управление тестами	Создание/редактирование тестов и вопросов разных типов	RTL, Supertest
Ролевая модель	Запрет доступа student к teacher/admin функциям	Supertest, middleware tests
Аналитика	Статистика теста, история результатов, экспорт PDF/Excel	Интеграционная проверка UI и API
Инфраструктура	Swagger, healthcheck, Docker Compose, CI/CD	Swagger tests, curl, docker compose, GitHub Actions
Фаззинг API	Нестандартные payload и проверка устойчивости endpoint	server/tests/fuzzing/fuzzing.js

Таблица 3.12 - Этапы CI/CD-пайплайна

Этап	Действие	Назначение
Checkout	Получение исходного кода из репозитория	Подготовка проекта к проверке
Setup Node.js	Установка Node.js 18 и настройка npm cache	Единая среда для frontend и backend
Install client	npm ci --prefix client	Установка зависимостей клиентской части
Install server	npm ci --prefix server	Установка зависимостей серверной части
Tests	npm test	Запуск автоматизированных тестов
Compose build	docker compose --env-file .env.example up -d --build	Сборка и запуск контейнеров
Backend healthcheck	curl http://localhost:5000/api/health	Проверка API-сервиса
Frontend check	curl http://localhost:3001	Проверка клиентского приложения
Nginx proxy check	curl http://localhost/api/health	Проверка проксирования API через Nginx
Logs on failure	docker compose logs --tail=200	Диагностика ошибок контейнеров
Cleanup	docker compose down -v	Остановка сервисов после проверки
Docker Hub publish	docker/build-push-action	Публикация backend и frontend образов

По результатам оценки приложение выполняет основные требования: разделяет пользователей по ролям, защищает операции изменения данных,

автоматически проверяет ответы, сохраняет результаты, предоставляет аналитику, документирует API, запускается в Docker Compose и проходит автоматизированные проверки.

К сильным сторонам реализации относится разделение frontend и backend, явная модель ролей, повторная валидация данных на сервере, использование JSON-полей для разных типов вопросов, нормализация ответов перед проверкой, хранение результатов с длительностью прохождения и наличие экспортируемой аналитики.

К ограничениям текущей версии относится отсутствие отдельного журнала административных действий, ограниченный набор типов вопросов и отсутствие встроенного механизма резервного копирования базы данных. Эти ограничения не мешают базовой работе системы, но важны для дальнейшего развития.

Рекомендуемые направления развития: добавить вопросы с открытым ответом и ручной проверкой, реализовать импорт тестов из Excel, добавить журнал действий администратора, настроить резервное копирование PostgreSQL, расширить отчёты преподавателя, добавить и реализовать уведомления пользователя о назначенных тестах.

ЗАКЛЮЧЕНИЕ

В ходе работы разработано клиент-серверное веб-приложение «Система автоматизированного тестирования и самопроверки знаний». Система реализует полный цикл электронного контроля знаний: создание тестов, настройку вопросов разных типов, прохождение студентом, автоматическую проверку, сохранение результата и просмотр аналитики преподавателем.

Практическая часть выполнена на React, Node.js, Express.js и PostgreSQL. Реализованы JWT-аутентификация, ролевая модель, REST API, административные функции, экспорт отчётности, Swagger, healthcheck, Docker Compose и CI/CD. Работоспособность подтверждена компонентными, модульными, API- и фаззинг-тестами, а приложение подготовлено к развёртыванию в production-среде.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Фаулер М. Архитектура корпоративных программных приложений. - Москва: Вильямс, 2019. - 544 с.
2. Richards M., Ford N. Fundamentals of Software Architecture: An Engineering Approach. - Sebastopol: O'Reilly Media, 2020. - 432 p.
3. Fielding R. Architectural Styles and the Design of Network-based Software Architectures: Doctoral dissertation. - Irvine: University of California, 2000. - 162 p.
4. React Documentation [Электронный ресурс]. - Режим доступа: <https://react.dev/> (дата обращения: 20.05.2026).
5. React Router Documentation [Электронный ресурс]. - Режим доступа: <https://reactrouter.com/> (дата обращения: 20.05.2026).
6. Redux Toolkit Documentation [Электронный ресурс]. - Режим доступа: <https://redux-toolkit.js.org/> (дата обращения: 20.05.2026).
7. Node.js Documentation [Электронный ресурс]. - Режим доступа: <https://nodejs.org/> (дата обращения: 20.05.2026).
8. Express.js Documentation [Электронный ресурс]. - Режим доступа: <https://expressjs.com/> (дата обращения: 20.05.2026).
9. Sequelize Documentation [Электронный ресурс]. - Режим доступа: <https://sequelize.org/> (дата обращения: 20.05.2026).
10. PostgreSQL Documentation [Электронный ресурс]. - Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 20.05.2026).
11. JSON Web Tokens [Электронный ресурс]. - Режим доступа: <https://jwt.io/> (дата обращения: 20.05.2026).
12. Swagger Documentation [Электронный ресурс]. - Режим доступа: <https://swagger.io/docs/> (дата обращения: 20.05.2026).
13. Render Documentation [Электронный ресурс]. - Режим доступа: <https://render.com/docs> (дата обращения: 20.05.2026).